

Spatial Data Science

Graduate – Elective (MSDS/MSCS/PhDCS)



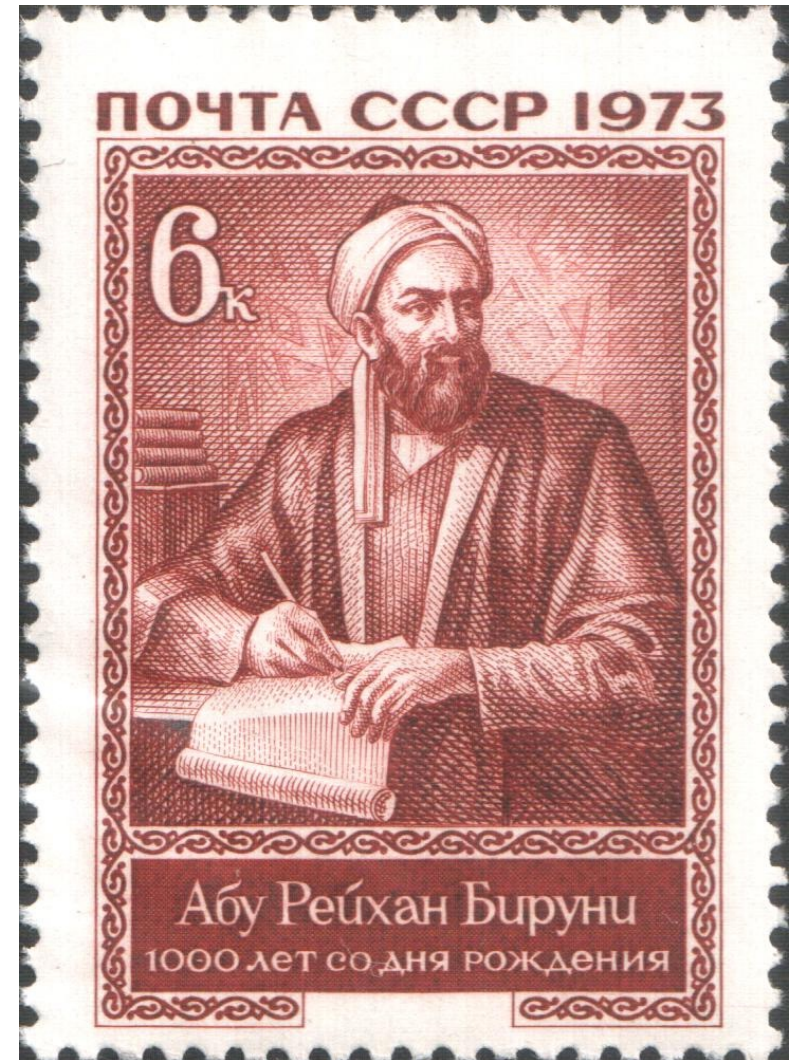
Remote Sensing
&
Spatial Analytics

Dr. sc. Muhammad Adnan Siddique
Assistant Professor
Information Technology University - ITU

Father of modern geodesy

Abu Rayhan Muhammad ibn Ahmad al-Biruni

(CE 973 –1050)



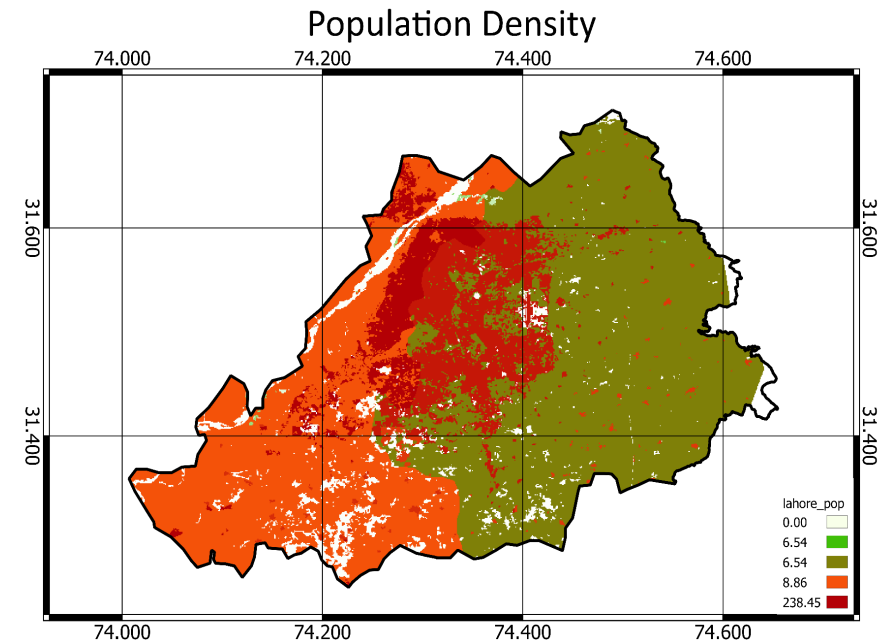
By The original uploader was Romanm at Slovenian Wikipedia. - Originally from sl.wikipedia; description page is/was here., Public Domain, <https://commons.wikimedia.org/w/index.php?curid=2590841>

Surfaces

Introduction on how to represent surfaces

Surfaces

- Surfaces are used to record data from a field data model.
- In theory, a field is a continuous surface and thus has an infinite number of locations at which it could be measured.
 - In reality however, fields are measured at a finite sample of locations that, to provide a sense of continuity and better conform with the field model, are uniformly structured across space.



Example

- To explore how Python represents surfaces, we will use an extract for the Brazilian city of Sao Paulo of a global population dataset.
 - This dataset records population counts in cells of the same dimensions uniformly covering the surface of the Earth.
- Our extract is available as a GeoTIF file, a variation of the TIF image format that includes geographic information.
 - We can use the `open_rasterio()` method from the `xarray` package to read in the GeoTIF:

```
pop = xarray.open_rasterio("../data/ghsl/ghsl_sao_paulo.tif")
```

```
type(pop)
```

```
xarray.core.dataarray.DataArray
```



pop.coords

- In `xarray`, the coordinates can be retrieved from our `DataArray` through the `coords` attribute:

```
pop.coords
```

Coordinates:

```
* band      (band) int64 1
* y         (y) float64 -2.822e+06 -2.822e+06 ... -2.926e+06
* x         (x) float64 -4.482e+06 -4.482e+06 ... -4.365e+06
```

- Our surface has three dimensions: `x`, `y`, and `band`.
 - The former two track the latitude and longitude that each cell in our population grid covers. The third one has a single value (1).
 - A third dimension can be very useful.
 - For example, an optical color image may have three bands: red, blue, and green.
 - More powerful sensors may pick up additional bands, such as near infrared (NIR) or even radio bands.
 - Or, a surface measured over time,



pop.attrs

- An `xarray.DataArray` object contains additional information about the values stored under the `attrs` attribute:

```
pop.attrs
```

```
{'transform': (250.0, 0.0, -4482000.0, 0.0, -250.0, -2822000.0),  
 'crs': '+proj=moll +lon_0=0 +x_0=0 +y_0=0 +datum=WGS84 +units=m',  
 'res': (250.0, 250.0),  
 'is_tiled': 0,  
 'nodatavals': (-200.0,),  
 'scales': (1.0,),  
 'offsets': (0.0,),  
 'AREA_OR_POINT': 'Area',  
 'grid_mapping': 'spatial_ref'}
```



- Thus, our `DataArray` has three dimensions:

```
pop.shape
```

```
(1, 416, 468)
```

- A common operation will be to reduce this to only the two geographic ones. We can do this with the `sel` operator, which allows us to select data by the value of their coordinates:





```
pop.sel(band=1)
```

[9]

✓ 0.6s

...

xarray.DataArray (y: 416, x: 468)



[194688 values with dtype=float32]

▼ Coordinates:

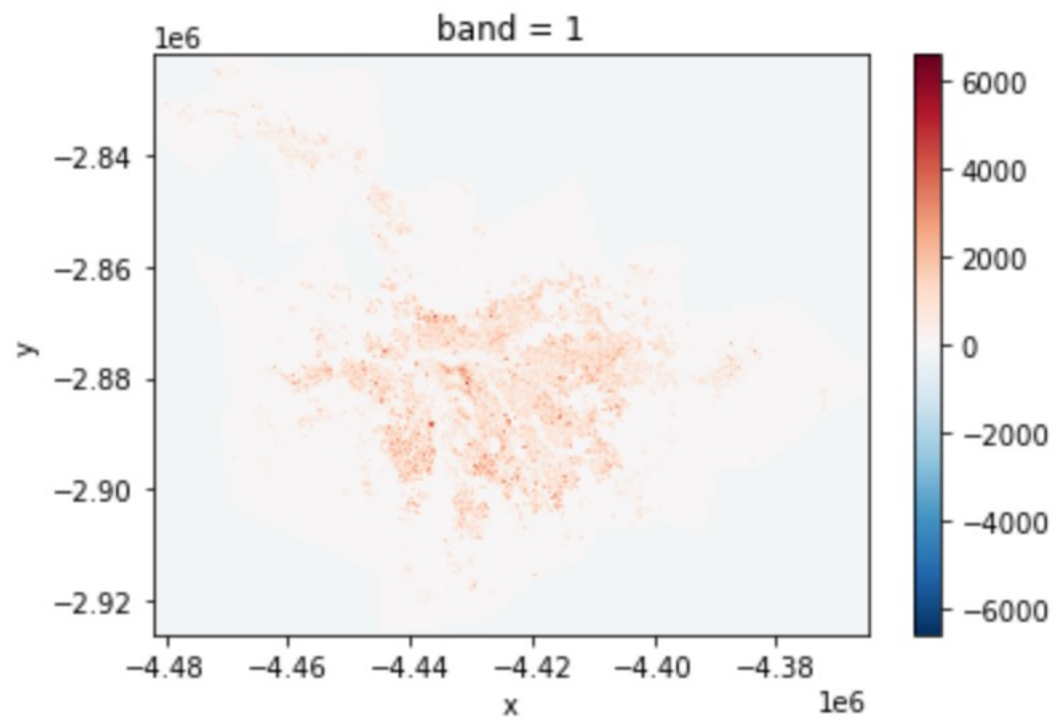
band	()	int64	1		
y	(y)	float64	-2.822e+06 ... -2.926e+06		
x	(x)	float64	-4.482e+06 ... -4.365e+06		

► Indexes: (2)

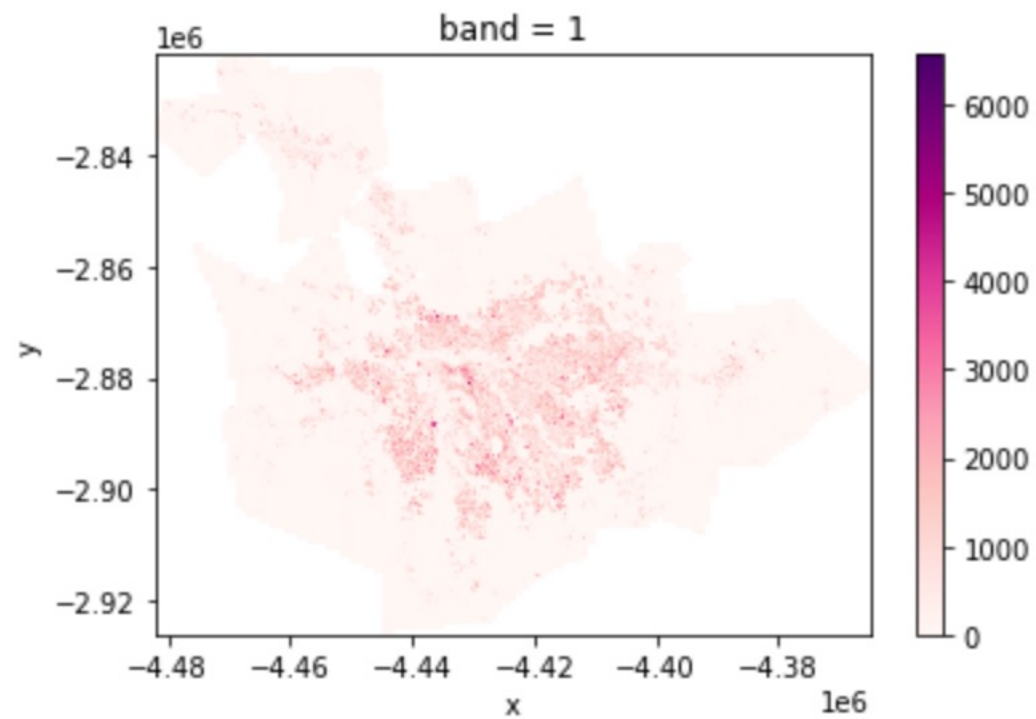
▼ Attributes:

transform : (250.0, 0.0, -4482000.0, 0.0, -250.0, -2822000.0)
crs : +proj=moll +lon_0=0 +x_0=0 +y_0=0 +datum=WGS84 +units=m +no_defs=True
res : (250.0, 250.0)
is_tiled : 0
nodatavals : (-200.0,)
scales : (1.0,)
offsets : (0.0,)
AREA_OR_POINT : Area
grid_mapping : spatial_ref

```
pop.sel(band=1).plot();
```



```
pop.where(pop != -200).sel(band=1).plot(cmap="RdPu");
```

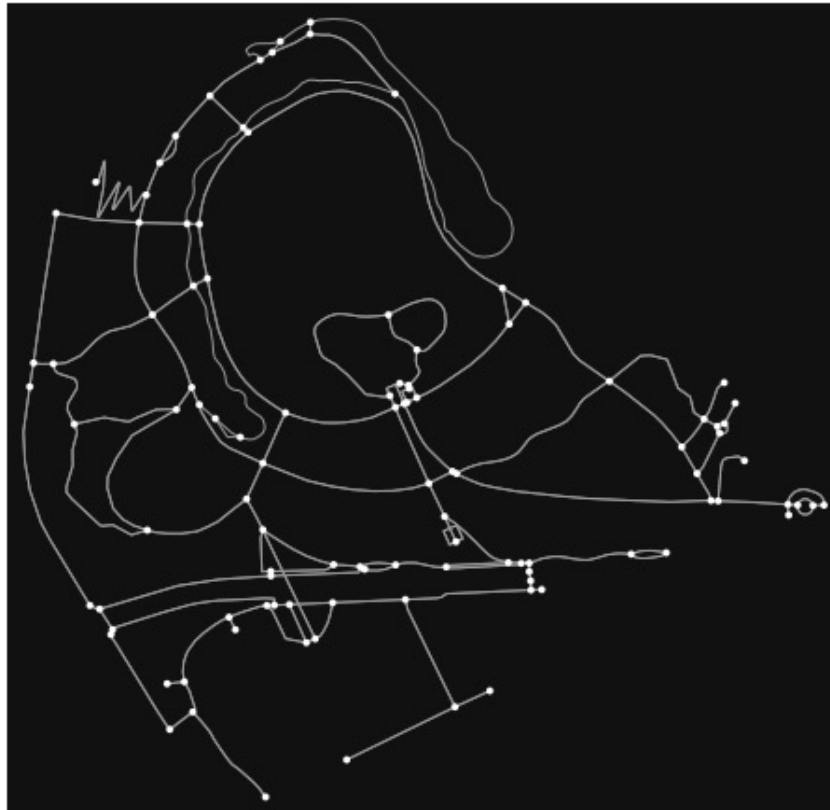


Spatial Graphs

Introduction

Spatial Graphs

- Spatial graphs store connections between objects through space.
 - These connections may derive from geographical topology (e.g., contiguity), distance, or more sophisticated dimensions such as interaction flows (e.g., commuting, trade, communication).



Spatial Graphs

- Compared to geographic tables and surfaces, spatial graphs are rather different.
 - First, in most cases they do not record measurements about a given phenomena, but instead focus on connections, on storing relationships between objects as they are facilitated (or impeded in their absence) by space.
 - Second, because of this relational nature, the data are organized in a more unstructured fashion: while one sample may be connected to only one other sample, another one can display several links.



Spatial graphs

```
graph = osmnx.graph_from_place("Yoyogi Park, Shibuya, Tokyo, Japan")
```

```
type(graph)
```

```
networkx.classes.multidigraph.MultiDiGraph
```

```
osmnx.plot_graph(graph);
```

```
len(graph.nodes)
```

```
106
```

and 287 edges (streets) that connect them:

```
len(graph.edges)
```

```
287
```



Spatial graphs

- Each of these elements can be queried to obtain more information such as the location and ID of a node:

```
graph.nodes[1520546819]
```

```
{'y': 35.6711267, 'x': 139.6925951, 'street_count': 4}
```

```
graph.edges[(1520546819, 3010293622, 0)]
```

```
{'osmid': 138670840,  
  'highway': 'footway',  
  'oneway': False,  
  'length': 59.113,  
  'geometry': <shapely.geometry.linestring.LineString at 0x7f3123456190>}
```



Hybrids

- We have just seen how geographic tables, surfaces, and networks map onto `GeoDataFrame`, `DataArray` and `Graph` objects in Python, respectively.
- However, while the conventional pairings are well-used, there are others in active use and many more to yet be developed.



Surfaces as tables

- Machine learning packages such as `scikit-learn`, or some spatial analytics (such as most methods in the Pysal family of packages) are designed around this data structure.
- Converting our surfaces into tables thus allows us to plug into a much wider suite of (potentially) efficient tools and techniques.



Surfaces as tables

```
surface = xarray.open_rasterio("../data/ghsl/ghsl_sao_paulo.tif")
```

```
t_surface = surface.to_series()
```

- The resulting object is a `pandas.Series` object indexed on each of the dimensions of the original `DataArray`:

```
t_surface.head()
```

band	y	x
1	-2822125.0	-4481875.0 -200.0
		-4481625.0 -200.0
		-4481375.0 -200.0
		-4481125.0 -200.0
		-4480875.0 -200.0

dtype: float32



Surfaces as tables

- The table we have built has no geometries associated with it, only rows representing pixels.

```
t_surface = t_surface.reset_index().rename(columns={0: "Value"})
```

▶ ▾

```
type(t_surface)
```

[10] ✓ 0.0s

... pandas.core.frame.DataFrame

```
t_surface.head()
```

[11] ✓ 0.1s

...

	band	y	x	Value
0	1	-2822125.0	-4481875.0	-200.0
1	1	-2822125.0	-4481625.0	-200.0
2	1	-2822125.0	-4481375.0	-200.0
3	1	-2822125.0	-4481125.0	-200.0
4	1	-2822125.0	-4480875.0	-200.0

